

4-Puzzle Lineal — BFS y DFS

Documentación Técnica

Inteligencia Artificial

Índice

1. Descripción del Problema	2
1.1. Operadores	2
2. BFS — Búsqueda por Amplitud	2
3. DFS — Búsqueda por Profundidad	2
4. Generación de Hijos	3
5. Visualización Animada	3
6. Comparativa BFS vs DFS en este problema	3

1. Descripción del Problema

El **4-Puzzle Lineal** es un problema de búsqueda donde se tiene un arreglo de 4 elementos que deben quedar en orden ascendente [1, 2, 3, 4] usando únicamente tres operadores de intercambio.

1.1. Operadores

Operador	Acción	Posiciones
Izquierdo	Intercambia pos 1↔2	índices 0 y 1
Central	Intercambia pos 2↔3	índices 1 y 2
Derecho	Intercambia pos 3↔4	índices 2 y 3

El espacio de búsqueda máximo es $4! = 24$ estados únicos.

2. BFS — Búsqueda por Amplitud

Explora nivel a nivel usando una cola FIFO. Garantiza la ruta con el **mínimo número de movimientos**.

```
1 int[] resolverBFS(int[] estadoInicial) {
2     Queue<Nodo> cola = new LinkedList<>();
3     Set<String> visitados = new HashSet<>();
4     cola.add(new Nodo(estadoInicial, null));
5     visitados.add(Arrays.toString(estadoInicial));
6
7     while (!cola.isEmpty()) {
8         Nodo actual = cola.poll();
9         if (esMeta(actual.estado)) return actual;
10
11         for (int[] hijo : generarHijos(actual.estado)) {
12             String clave = Arrays.toString(hijo);
13             if (!visitados.contains(clave)) {
14                 visitados.add(clave);
15                 cola.add(new Nodo(hijo, actual));
16             }
17         }
18     }
19     return null;
20 }
```

3. DFS — Búsqueda por Profundidad

Explora profundamente usando una pila LIFO. No garantiza la solución óptima pero puede ser más rápido en algunos casos.

```
1 int[] resolverDFS(int[] estadoInicial) {
2     Stack<Nodo> pila = new Stack<>();
3     Set<String> visitados = new HashSet<>();
4     pila.push(new Nodo(estadoInicial, null));
```

```

5
6     while (!pila.isEmpty()) {
7         Nodo actual = pila.pop();
8         String clave = Arrays.toString(actual.estado);
9         if (visitados.contains(clave)) continue;
10        visitados.add(clave);
11
12        if (esMeta(actual.estado)) return actual;
13        for (int[] hijo : generarHijos(actual.estado))
14            pila.push(new Nodo(hijo, actual));
15    }
16    return null;
17 }

```

4. Generación de Hijos

Cada estado genera hasta 3 hijos aplicando los 3 operadores:

```

1 List<int[]> generarHijos(int[] estado) {
2     List<int[]> hijos = new ArrayList<>();
3     for (int i = 0; i < 3; i++) { // 3 intercambios
4         int[] hijo = estado.clone();
5         int tmp = hijo[i];
6         hijo[i] = hijo[i+1];
7         hijo[i+1] = tmp;
8         hijos.add(hijo);
9     }
10    return hijos;
11 }

```

5. Visualización Animada

Una vez obtenida la solución, se reconstruye el camino desde el nodo meta hasta el inicial siguiendo los punteros padre. La animación se implementa con `javax.swing.Timer`:

- Cada 500ms se muestra el siguiente estado del camino.
- El tablero visual actualiza los colores de las celdas intercambiadas.
- Se muestra el número de paso actual sobre el total.

6. Comparativa BFS vs DFS en este problema

Característica	BFS	DFS
Solución óptima	Sí	No siempre
Memoria usada	Mayor	Menor
Velocidad	Similar	Similar
Espacio explorado	Completo por nivel	Profundidad primero